



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 20

“Proyecto 3: Ping-Pong”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 24 de abril de 2026

INTRODUCCIÓN

Matriz de led 7x5

Una matriz de LEDs de 7x5 es un dispositivo de visualización compuesto por 35 diodos emisores de luz organizados en 7 filas y 5 columnas. Es una de las configuraciones más populares en electrónica para mostrar caracteres alfanuméricos, símbolos y gráficos pequeños de forma clara y eficiente.

Configuración Interna (Ánodo y Cátodo)

Para ahorrar pines, los LEDs no se conectan de forma individual (lo que requeriría 35 pines más tierra). En su lugar, se comparten conexiones:

- **Matriz de Ánodo Común en Filas:** Todos los ánodos de los LEDs en una misma fila están conectados entre sí.
- **Matriz de Cátodo Común en Columnas:** Todos los cátodos de los LEDs en una misma columna están conectados entre sí.

(También existe la configuración inversa, dependiendo del fabricante).

El Método de Multiplexación

Debido a esta estructura compartida, no puedes encender todos los LEDs que quieras simultáneamente de forma estática. Se utiliza la multiplexación por barrido:

1. **Activación de Columna:** Se energiza la primera columna (C1).
2. **Envío de Datos:** Se envían los estados (encendido/apagado) de los 7 LEDs de esa columna a través de las filas (R1-R7).
3. **Persistencia de la Visión (POV):** Se apaga la C1 y se repite el proceso con la C2, C3, C4 y C5 a una velocidad superior a **60 Hz**.
4. **Resultado:** El ojo humano no percibe el parpadeo y ve una imagen estable y completa.

Hardware de Control

Controlar 12 pines (7 filas + 5 columnas) puede agotar rápidamente los puertos de un microcontrolador. Por ello, es común usar:

- **Registros de Desplazamiento:** Permiten controlar las filas usando solo 3 pines del microcontrolador.
- **Decodificadores:** Para seleccionar de forma rápida qué columna activar.
- **Drivers de Corriente:** Dado que una fila puede tener 5 LEDs encendidos a la vez, el microcontrolador podría no soportar la corriente necesaria; estos chips actúan como amplificadores.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 20, el circuito a armar es el siguiente:

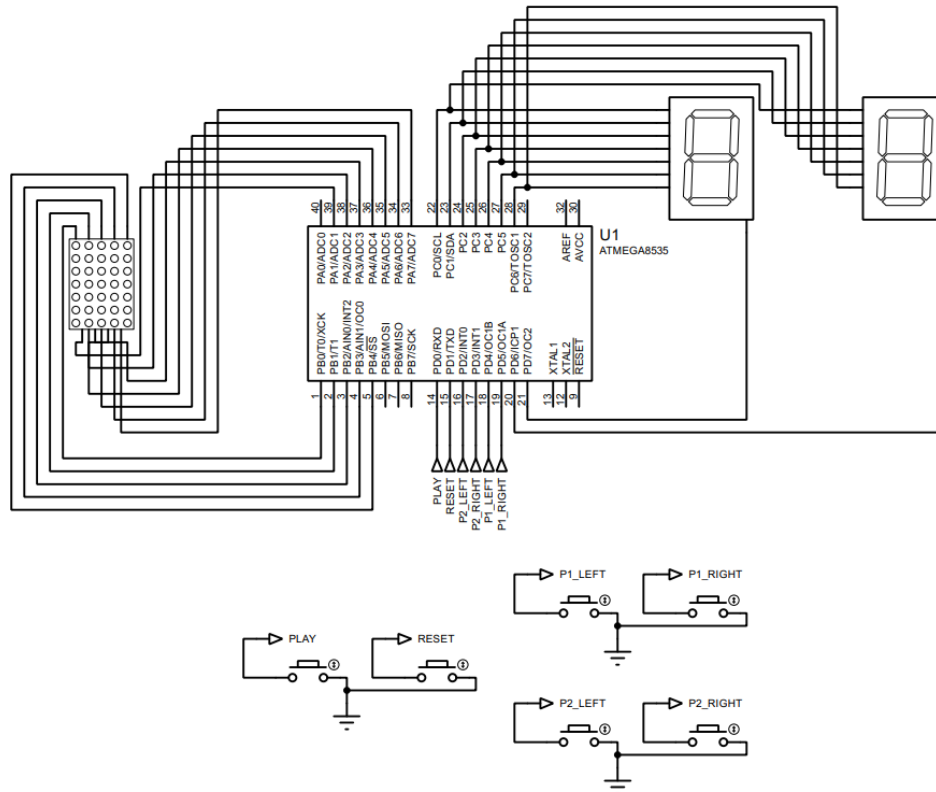


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen, el registro A (PA1-PA7), B (PB0-PB4), C (PC0-PC6) como de salida mientras que parte del registro D se configura como de salida (PD6-PD7) y el resto como de entrada (PD0-PD5) activando las resistencias de pull-up.

Ports Settings

PORTA Port B Port C Port D

Data Direction	Pullup/Output Value
Bit 0 Out	0 Bit 0
Bit 1 Out	0 Bit 1
Bit 2 Out	0 Bit 2
Bit 3 Out	0 Bit 3
Bit 4 Out	0 Bit 4
Bit 5 Out	0 Bit 5
Bit 6 Out	0 Bit 6
Bit 7 Out	0 Bit 7

Imagen 2. Configuración del puerto A.

Ports Settings

PORTA Port B Port C Port D

Data Direction	Pullup/Output Value
Bit 0 Out	0 Bit 0
Bit 1 Out	0 Bit 1
Bit 2 Out	0 Bit 2
Bit 3 Out	0 Bit 3
Bit 4 Out	0 Bit 4
Bit 5 In	T Bit 5
Bit 6 In	T Bit 6
Bit 7 In	T Bit 7

Imagen 3. Configuración del puerto B.

Ports Settings

PORTA Port B Port C Port D

Data Direction	Pullup/Output Value
Bit 0 Out	0 Bit 0
Bit 1 Out	0 Bit 1
Bit 2 Out	0 Bit 2
Bit 3 Out	0 Bit 3
Bit 4 Out	0 Bit 4
Bit 5 Out	0 Bit 5
Bit 6 Out	0 Bit 6
Bit 7 In	T Bit 7

Imagen 4. Configuración del puerto C.

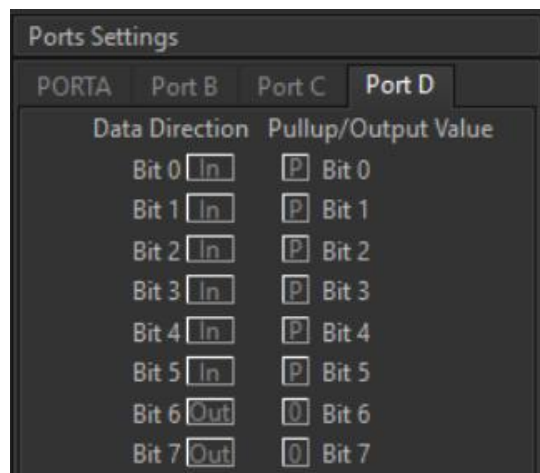


Imagen 5. Configuraci3n del puerto D.

C3digo del circuito de CodeVision:

```

1  /******
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4  Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr3ctica 20 'Juego de Ping-Ping'
8  Version : 1.0
9  Date    : 20/04/2026
10 Author  : Garc3a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type      : ATmega8535
16 Program type   : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model   : Small
19 External RAM size : 0
20 Data Stack size : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>
26 #include <stdlib.h>

```

```

29 #define inP1_left PIND.4
30 #define inP2_right PIND.3
31 #define inP2_left PIND.2
32 #define in_reset PIND.1
33 #define in_play PIND.0
34 #define TOP_LIMIT 0x18
35 #define BOTTOM_LIMIT 0x03
36
37
38 // Declare your global variables here
39 unsigned char num[7] = {0x06, 0x00, 0x00, 0x04, 0x00, 0x00, 0x0c};
40 bit botonp;
41 bit botona, game = 1;
42 bit dir; // direcci3n izquierda: 0, derecha: 1
43 int opc, index_ball;
44 int dir_ball, timer; // supizq: 0, sup: 1, supder: 2, infder: 3, inf: 4, indizq: 5
45 unsigned char scoreP1, scoreP2;
46 const char tabla7segmentos[10] = {0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};
47
48 void verVector(unsigned char myVector[7]) {
49     unsigned char i, v = 0x01;
50
51     for (i = 0; i < 7; i++) {
52         PORTA = ~v << 1; // inicia desde el bit A1, por eso el corrimiento hacia la izquierda
53         PORTB = myVector[i];
54         delay_ms(3);
55         v = v << 1;
56     }
57 }

```

```

59 void jugador1_move(int my_opc) {
60     unsigned char temp;
61
62     if (my_opc == 1) {
63         temp = num[0] << 1;
64         if (temp > TOP_LIMIT) {
65             num[0] = TOP_LIMIT;
66         } else {
67             num[0] = temp;
68         }
69     } else {
70         temp = num[0] >> 1;
71         if (temp < BOTTOM_LIMIT) {
72             num[0] = BOTTOM_LIMIT;
73         } else {
74             num[0] = temp;
75         }
76     }
77 }

```

```
79 void jugador2_move(int my_opc) {
80     unsigned char temp;
81
82     if (my_opc == 3) {
83         temp = num[6] << 1;
84         if (temp > TOP_LIMIT) {
85             num[6] = TOP_LIMIT;
86         } else {
87             num[6] = temp;
88         }
89     } else {
90         temp = num[6] >> 1;
91         if (temp < BOTTOM_LIMIT) {
92             num[6] = BOTTOM_LIMIT;
93         } else {
94             num[6] = temp;
95         }
96     }
97 }
```

```
99 void checa_boton(void) {
100     if (inP1_right == 0) {
101         botona = 0;
102         opc = 1;
103     } else if (inP1_left == 0) {
104         botona = 0;
105         opc = 2;
106     } else if (inP2_right == 0) {
107         botona = 0;
108         opc = 3;
109     } else if (inP2_left == 0) {
110         botona = 0;
111         opc = 4;
112     } else if (in_reset == 0) {
113         botona = 0;
114         opc = 5;
115     } else if (in_play == 0) {
116         botona = 0;
117         opc = 6;
118     } else {
119         botona = 1;
120     }
}
```

```
122   if (botonp && !botona) { // hubo cambio de flanco de 1 a 0
123       if (opc == 1 || opc == 2) {
124           jugador1_move(opc);
125       } else if (opc == 3 || opc == 4) {
126           jugador2_move(opc);
127       } else if (opc == 5) {
128           scoreP1 = 0;
129           scoreP2 = 0;
130           game = 1;
131       } else {
132           game = !game;
133       }
134
135       delay_ms(8); // para eliminar rebotes
136   }
137
138   if (!botonp && botona) { // hubo cambio de flanco de 0 a 1
139       delay_ms(8); // para eliminar rebotes
140   }
141
142   botonp = botona;
143   opc = -1;
144 }
```

```
146 void recorrerVector_der() {
147     unsigned int r_num = rand() % 6;
148
149     if (index_ball + 1 == 6) {
150         dir = 0;
151         if ((num[index_ball + 1] & num[index_ball]) == 0) {
152             scoreP2++;
153             num[index_ball] = 0x00;
154             index_ball = 3;
155             num[index_ball] = 0x04;
156         } else {
157             if (r_num < 3) {
158                 dir_ball = r_num;
159             } else {
160                 dir_ball = 1;
161             }
162         }
163     } else {
164         num[index_ball + 1] = num[index_ball];
165         num[index_ball] = 0x00;
166         index_ball++;
167     }
168 }
```



```
170 void recorrerVector_izq() {
171     unsigned int r_num = rand() % 6;
172
173     if (index_ball - 1 == 0) {
174         dir = 1;
175         if ((num[index_ball - 1] ^ num[index_ball]) > num[index_ball - 1]) {
176             scoreP1++;
177             num[index_ball] = 0x00;
178             index_ball = 3;
179             num[index_ball] = 0x04;
180         } else {
181             if (r_num >= 3) {
182                 dir_ball = r_num;
183             } else {
184                 dir_ball = 4;
185             }
186         }
187     } else {
188         num[index_ball - 1] = num[index_ball];
189         num[index_ball] = 0x00;
190         index_ball--;
191     }
192 }
```

```
194 void pelota_move() {
195     unsigned char temp;
196
197     if (dir) { // abajo
198         switch (dir_ball) {
199             case 3:
200                 temp = num[index_ball] >> 1;
201                 if (temp != 0x00) {
202                     num[index_ball] = temp;
203                     recorrerVector_der();
204                 } else {
205                     dir_ball = 5;
206                     recorrerVector_der();
207                 }
208                 break;
209             case 4:
210                 recorrerVector_der();
211                 break;
212             case 5:
213                 temp = num[index_ball] << 1;
214                 if (temp != 0x10) {
215                     num[index_ball] = temp;
216                     recorrerVector_der();
217                 } else {
218                     dir_ball = 3;
219                     recorrerVector_der();
220                 }
221                 break;
222             default:
223                 dir_ball = 4;
224                 break;
225         }
226     }
```

```

226 |     } else {
227 |         switch (dir_ball) {
228 |             case 2:
229 |                 temp = num[index_ball] >> 1;
230 |                 if (temp != 0x00) {
231 |                     num[index_ball] = temp;
232 |                     recorreVector_izq();
233 |                 } else {
234 |                     dir_ball = 0;
235 |                     recorreVector_izq();
236 |                 }
237 |                 break;
238 |             case 1:
239 |                 recorreVector_izq();
240 |                 break;
241 |             case 0:
242 |                 temp = num[index_ball] << 1;
243 |                 if (temp != 0x10) {
244 |                     num[index_ball] = temp;
245 |                     recorreVector_izq();
246 |                 } else {
247 |                     dir_ball = 2;
248 |                     recorreVector_izq();
249 |                 }
250 |                 break;
251 |             default:
252 |                 dir_ball = 1;
253 |                 break;
254 |         }
255 |     }
256 | }

```

```

258 | void verMarcador() {
259 |     PORTD.7 = 0;
260 |     PORTC = 0x00;
261 |     PORTD.6 = 1;
262 |     PORTC = tabla7segmentos[scoreP1];
263 |     delay_ms(2);
264 |
265 |     PORTD.6 = 0;
266 |     PORTC = 0x00;
267 |     PORTD.7 = 1;
268 |     PORTC = tabla7segmentos[scoreP2];
269 |     delay_ms(2);
270 | }
271 |
272 | void main(void) {
273 |     // Declare your local variables here
274 |     opc = -1;
275 |     index_ball = 3;
276 |     timer = 0;
277 |     dir = 1;
278 |     scoreP1 = 0;
279 |     scoreP2 = 0;
280 |     dir_ball = 1;
281 |     srand(1);
282 |
283 |     // Input/Output Ports initialization
284 |     // Port A initialization
285 |     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=In
286 |     DDRA=(1<<DDA7) | (1<<DDA6) | (1<<DDA5) | (1<<DDA4) | (1<<DDA3) | (1<<DDA2) | (1<<DDA1) | (0<<DDA0);
287 |     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=1
288 |     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

```

```

290 // Port B initialization
291 // Function: Bit7=In Bit6=In Bit5=In Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
292 DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (1<<DDB4) | (1<<DDB3) | (1<<DDB2) | (1<<DDB1) | (1<<DDB0);
293 // State: Bit7=T Bit6=T Bit5=T Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
294 PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
295
296 // Port C initialization
297 // Function: Bit7=In Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
298 DDRC=(0<<DDC7) | (1<<DDC6) | (1<<DDC5) | (1<<DDC4) | (1<<DDC3) | (1<<DDC2) | (1<<DDC1) | (1<<DDC0);
299 // State: Bit7=T Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
300 PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
301
302 // Port D initialization
303 // Function: Bit7=Out Bit6=Out Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
304 DDRD=(1<<DDD7) | (1<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
305 // State: Bit7=0 Bit6=0 Bit5=P Bit4=P Bit3=P Bit2=P Bit1=P Bit0=P
306 PORTD=(0<<PORTD7) | (0<<PORTD6) | (1<<PORTD5) | (1<<PORTD4) | (1<<PORTD3) | (1<<PORTD2) | (1<<PORTD1) | (1<<PORTD0);
307
308 // Timer/Counter 0 initialization
309 // Clock source: System Clock
310 // Clock value: Timer 0 Stopped
311 // Mode: Normal top=0xFF
312 // OC0 output: Disconnected
313 TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
314 TCNT0=0x00;
315 OCR0=0x00;

```

```

317 // Timer/Counter 1 initialization
318 // Clock source: System Clock
319 // Clock value: Timer1 Stopped
320 // Mode: Normal top=0xFFFF
321 // OC1A output: Disconnected
322 // OC1B output: Disconnected
323 // Noise Canceler: Off
324 // Input Capture on Falling Edge
325 // Timer1 Overflow Interrupt: Off
326 // Input Capture Interrupt: Off
327 // Compare A Match Interrupt: Off
328 // Compare B Match Interrupt: Off
329 TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
330 TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
331 TCNT1H=0x00;
332 TCNT1L=0x00;
333 ICR1H=0x00;
334 ICR1L=0x00;
335 OCR1AH=0x00;
336 OCR1AL=0x00;
337 OCR1BH=0x00;
338 OCR1BL=0x00;
339
340 // Timer/Counter 2 initialization
341 // Clock source: System Clock
342 // Clock value: Timer2 Stopped
343 // Mode: Normal top=0xFF
344 // OC2 output: Disconnected
345 ASSR=0<<AS2;
346 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
347 TCNT2=0x00;
348 OCR2=0x00;

```

```

350 // Timer(s)/Counter(s) Interrupt(s) initialization
351 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
352
353 // External Interrupt(s) initialization
354 // INT0: Off
355 // INT1: Off
356 // INT2: Off
357 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
358 MCUCSR=(0<<ISC2);
359
360 // USART initialization
361 // USART disabled
362 UCSRB=(0<<RXCE) | (0<<TXCE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);
363
364 // Analog Comparator initialization
365 // Analog Comparator: Off
366 // The Analog Comparator's positive input is
367 // connected to the AIN0 pin
368 // The Analog Comparator's negative input is
369 // connected to the AIN1 pin
370 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
371 SFIOR=(0<<ACME);
372
373 // ADC initialization
374 // ADC disabled
375 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
376
377 // SPI initialization
378 // SPI disabled
379 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
380
381 // TWI initialization
382 // TWI disabled
383 TWCR=(0<<TWIEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

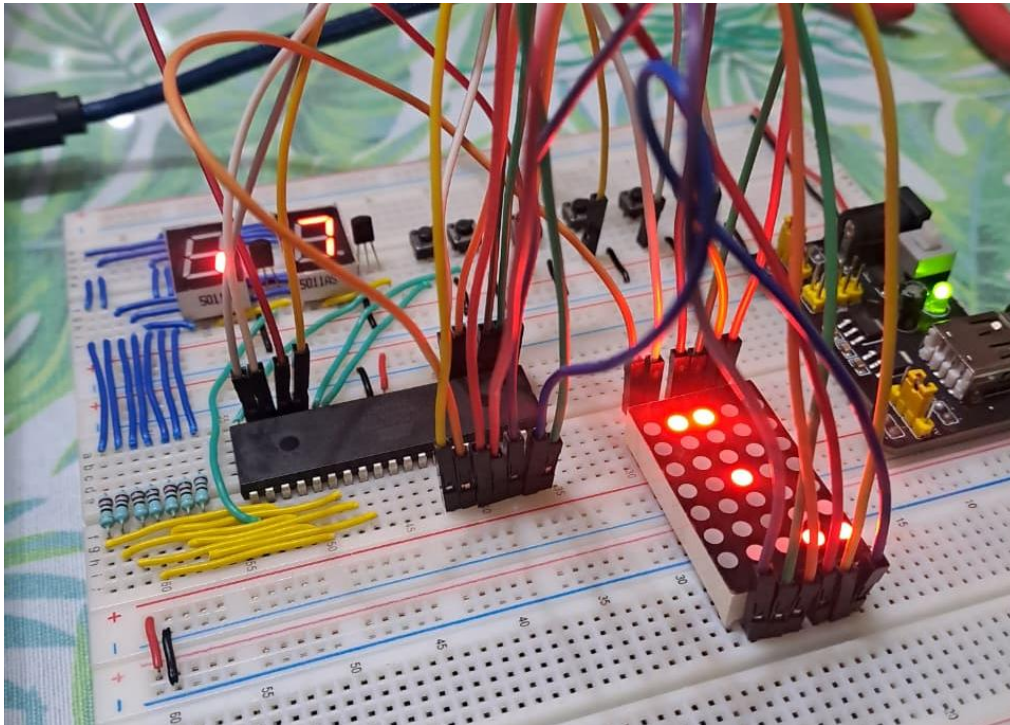
```

```

385 while (1) {
386     // Place your code here
387     if (scoreP1 >= 10 || scoreP2 >= 10) {
388         game = 1;
389     }
390
391     checa_boton();
392
393     if (!game) {
394         if (timer > 5) {
395             timer = 0;
396             pelota_move();
397         } else {
398             timer++;
399         }
400     }
401
402     verVector(num);
403     verMarcador();
404 }
405 }

```


Circuito físico



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Este programa demuestra cómo el microcontrolador ATMEGA8535 puede implementar un sistema interactivo en tiempo real, integrando lectura de entradas, control de salidas y lógica de juego. A través de estructuras simples como arreglos y operaciones binarias, se simula el comportamiento del juego Ping Pong, incluyendo movimiento, colisiones y puntuación. Además, se aplican conceptos clave de sistemas embebidos como multiplexación, control de rebotes y manejo eficiente de recursos, evidenciando cómo hardware y software trabajan en conjunto para crear aplicaciones dinámicas.

Entre lo más destacado de la práctica se encuentra la aleatoriedad de la pelota a lo largo de la matriz 5x7. Esta se implementa mediante la función `rand()` para generar números entre 0 y 5 y así modificar su dirección tras cada rebote con la paletas introduciendo estos números como variaciones en su rebote, sin embargo como en el código se usa `srand(1)`, es decir, se usa una semilla fija (genera la misma secuencia de números), por lo que el comportamiento de la pelota se vuelve determinista, es por eso que en la ejecución del juego pareciese que la pelota es aleatoria pero después se logra percibir que esta sigue la misma secuencia de movimientos.

Para poder mejorar dicha “aleatoriedad” de la pelota necesitamos una fuente de variaciones dinámicas, como el tiempo de ejecución `srand(timer)`, el ruido del hardware, es decir, leer un pin analógico sin conectar `srand(ADC_value)` lo que introduce variaciones reales

del entorno físico o usar el tiempo en el que alguno de los jugadores presiona un botón ya que este nunca será igual. La implementación de alguna de estas mejoras permitirá generar trayectorias más impredecibles y realistas para la pelota.

- **Meléndez Macedonio Rodrigo**

Realizar esta práctica nos permitió consolidar los conocimientos adquiridos sobre el desarrollo de sistemas interactivos en tiempo real utilizando el microcontrolador ATMEGA8535. La implementación del juego Ping-Pong en un entorno físico no solo requirió una lógica de control estructurada para gestionar las colisiones, el movimiento de la pelota y el puntaje, sino también un manejo eficiente de los recursos del hardware mediante la multiplexación y el control de rebotes en los botones.

Asimismo, la experiencia de trabajar con la matriz de LEDs de 5x7 y los displays de siete segmentos puso de manifiesto la importancia de la sincronización entre el software y el hardware para lograr una visualización fluida y dinámica. Por otro lado, el análisis del comportamiento de la pelota nos permitió comprender las limitaciones del software al generar aleatoriedad con una semilla fija, lo que subraya la necesidad de integrar variables del entorno físico para lograr un sistema verdaderamente impredecible. En conclusión, este proyecto demostró exitosamente cómo la correcta integración de la lógica de programación y las etapas de visualización permiten crear aplicaciones embebidas complejas y funcionales.

BIBLIOGRAFÍA

(s.f.). *Multiplexing*. All About Circuits. Recuperado el 28 de abril de 2026.
<https://www.allaboutcircuits.com/>

(s.f.). *Matriz de LEDs*. Luis Llamas. Recuperado el 28 de abril de 2026.
<https://www.luisllamas.es/>

(s.f.). *Pong (Atari 1972)*. Wikipedia. Recuperado el 28 de abril de 2026.
<https://es.wikipedia.org/wiki/Pong>